



IGCSE Computer Science CIE

8. Programming

CONTENTS

8.1 Programming Concepts

Data Types

Variables & Constants

Input & Output

Sequence

Selection

Iteration

Totalling & Counting

String Handling

Arithmetic, Logical & Boolean Operators

Nested Statements

Procedures & Functions

Local & Global Variables

Library Routines

Maintaining Programs

8.2 Arrays

Arrays

Using Arrays

8.3 File Handling

File Handling

8.2 Arrays

Arrays

YOUR NOTES



1D Arrays

A one-dimensional (1D) array is a collection of items of the same data type.

- To declare an array in pseudocode, use the syntax:

```
DECLARE arrayName[n] OF dataType
```

- To declare an array in Python, use the syntax:

```
array_name = [0] * n
```

- In Java, use the syntax:

```
dataType[] arrayName = new dataType[n];
```

- In Visual Basic, use the syntax:

```
Dim arrayName(n) As dataType
```

To access elements in a 1D array, use the index, which can start at 0 or 1.

2D Arrays

A two-dimensional (2D) array is an array of arrays, creating a grid-like structure.

- To declare a 2D array in pseudocode using the syntax:

```
DECLARE arrayName[n][m] OF dataType
```

- In Python, use the syntax:

```
array_name = [[0] * m for _ in range(n)]
```

- In Java, use the syntax:

```
dataType[][] arrayName = new dataType[n][m];
```

- In Visual Basic, use the syntax:

```
Dim arrayName(n, m) As dataType
```

To access elements in a 2D array, use two indices: the row index and the column index.

The syntax for accessing elements is similar across languages, using square brackets:

- `[i]` for 1D arrays,
- and `[i][j]` for 2D arrays.

8.3 File Handling

File Handling

Why Use Files?

- Files provide a way to store data permanently, allowing programs to access and manipulate it later
- Files allow for organised storage of data, making it easier to find, access, and update
- They enable sharing of data between different programs and users
- They support data backup, ensuring data is not lost when a program or computer system shuts down
- They allow for large amounts of data to be stored and managed efficiently
- Data can be transported from one system to another

YOUR NOTES



Opening & Closing Files

- Common file operations include:
 - Opening: creates a connection between the file and the program
 - Reading: retrieves data from a file
 - Writing: saves new data to a file, overwriting existing content
 - Appending: adds new data to the end of a file without overwriting existing content
 - Closing: ends the connection between the file and the program, releasing system resources
- Single items of data can be numbers, characters, or other simple data types
- A line of text typically ends with a newline character or an end-of-line marker

Pseudocode example:

```
OPEN "file.txt"
```

```
filedata = READ data
```

```
PRINT filedata
```

```
CLOSE file
```

```
OPEN "file.txt"
```

```
WRITE data
```

```
CLOSE file
```

Python example:

```
with open("file.txt", "r") as file:
```

```
    data = file.read()
```

```
    print(data)
```

```
with open("file.txt", "w") as file:
```

```
    file.write(data)
```

Java example:

```
public static void main(String[] args) {
```

```
    String path = ("file.txt");
```

```
    try {
```

```
        FileReader f = new FileReader(path);
```

```
        BufferedReader reader = new BufferedReader(f);
```

```
        String data = reader.readLine();
```

```
        reader.close();
```

```
        BufferedWriter out = new BufferedWriter(f);
```

```
        out.write(data);
```

```
        out.close();
```

```
    } catch (Exception e) {
```

```
        System.err.println("No file found");
```

YOUR NOTES



```
}
```

```
}
```

Visual Basic example:

```
Dim File As New System.IO.StreamReader("file.txt")
```

```
Console.WriteLine(File.ReadLine())
```

```
File.Close()
```

```
Dim fileWrite As New System.IO.StreamWriter("file.txt")
```

```
fileWrite.WriteLine(data)
```

```
fileWrite.Close()
```



Exam Tip

- Remember to close files after using them

YOUR NOTES



8.2 Arrays

Using Arrays

Using Arrays

- Arrays are used to store and manage multiple values of the same data type efficiently
- They can be used for tasks such as storing student scores, calculating averages, and managing inventory data

Using variables in arrays

- Variables can be used as indexes in arrays to access and modify elements dynamically
- This is useful when iterating through the array using loops or performing calculations based on user input

What is the first index value?

- The first index can be either 0 or 1, depending on the programming language and personal preference
- Most programming languages, such as Python, Java, and Visual Basic, use 0 as the first index



Exam Tip

- In pseudocode, the first index can be either 0 or 1, so it is crucial to read the question to find out

Pseudocode example:

```
DECLARE scores[5] OF INTEGER
```

```
FOR i FROM 0 TO 4
```

```
    INPUT scores[i]
```

```
ENDFOR
```

Python example:

```
scores = [0] * 5
```

```
for i in range(5):
```

```
    scores[i] = int(input())
```

Java example:

```
int[] scores = new int[5];
```

```
Scanner scanner = new Scanner(System.in);
```

```
for (int i = 0; i < 5; i++) {
```

```
    scores[i] = scanner.nextInt();
```

```
}
```

YOUR NOTES



Visual Basic example:

```
Dim scores(5) As Integer
```

```
For i As Integer = 0 To 4
```

```
    console.WriteLine("Enter score: ")
```

```
    scores(i) = Console.ReadLine()
```

```
Next
```

YOUR NOTES



Worked Example

Declare an array to store customer names

[1]

```
CustomerNames[1:30] [1]
```

Declare the arrays to store each customer's date of birth, telephone number, email address and balance

[2]

- CustomerDOB[1:30]
- CustomerTelNo[1:30]
- CustomerEmail[1:30] [1]
- CustomerBalance[1:30] [1]

Using Iteration

- Loops can be used to write values into arrays, iterating through each element and assigning a value
- Iteration can also be used to read values from arrays, traversing through each element and performing operations or outputting the value
- Nested iteration is useful for working with multi-dimensional arrays, such as 2D arrays, where you need to iterate through rows and columns

Pseudocode example:

```
DECLARE scores[3][3] OF INTEGER
FOR i FROM 0 TO 2
  FOR j FROM 0 TO 2
    INPUT scores[i][j]
  ENDFOR
ENDFOR

FOR i FROM 0 TO 2
  FOR j FROM 0 TO 2
    OUTPUT scores[i][j]
  ENDFOR
ENDFOR
```

Python example:

```
scores = [[0] * 3 for _ in range(3)]
for i in range(3):
    for j in range(3):
        scores[i][j] = int(input())

for i in range(3):
    for j in range(3):
        print(scores[i][j])
```

Java example:

```
int[][] scores = new int[3][3];
Scanner scanner = new Scanner(System.in);
for (int i = 0; i < 3; i++) {
    for (int j = 0; j < 3; j++) {
        scores[i][j] = scanner.nextInt();
    }
}

for (int i = 0; i < 3; i++) {
    for (int j = 0; j < 3; j++) {
        System.out.print(scores[i][j] + " ");
    }
    System.out.println();
}
```

YOUR NOTES



Visual Basic example:

```
Dim scores(2, 2) As Integer
For i As Integer = 0 To 2
    For j As Integer = 0 To 2

        Console.WriteLine("Enter score: ")
        scores(i, j) = Console.ReadLine()
    Next
Next

For i As Integer = 0 To 2
    For j As Integer = 0 To 2
        Console.WriteLine(scores(i, j) & " ")
    Next
    Console.WriteLine()
Next
```



Exam Tip

- Keep track of your loop variables to avoid confusion and errors – it's best to give these clear names (row and column) rather than i and j



Worked Example

Write the pseudocode to output the contents of the arrays `Balance[ ]` and `Email [ ]` along with suitable messages

```
Count ← 0
REPEAT
    PRINT "Customer: ", Count, " Balance: ", Balance[Count], " Email:
    ",Email[Count]
    Count ← Count + 1
UNTIL Count = 30
```

You could also answer with a WHILE or FOR loop

YOUR NOTES



8.1 Programming Concepts

Data Types

Data Types

- A data type is a classification of data into groups according to the kind of data they represent
- Computers use different data types to represent different types of data in a program
- The basic data types include:
 - Integer: used to represent whole numbers, either positive or negative
 - Examples: 10, -5, 0
 - Real: used to represent numbers with a fractional part, either positive or negative
 - Examples: 3.14, -2.5, 0.0
 - Char: used to represent a single character such as a letter, digit or symbol
 - Examples: 'a', 'B', '5', '\$'
 - String: used to represent a sequence of characters
 - Examples: "Hello World", "1234", "@#\$\$"
 - Boolean: used to represent true or false values
 - Examples: True, False

We can declare variables as follows:

	Python	Java	Visual Basic
Syntax	variable_name = value	data_type variable_name;	Dim variable_name As data_type
Example	x = 5	int x	Dim x As Integer

It is important to choose the correct data type for a given situation to ensure accuracy and efficiency in the program.

YOUR NOTES





Worked Example

Name and describe the most appropriate programming data type for each of the examples of data given. Each data type must be different

[6]

Data: 83

Data type name: **Integer** [1]

Data type description: **The number is a whole number** [1]

Data: myemail@savemyexams.co.uk

Data type name: **String** [1]

Data type description: **It is a group of characters** [1]

Data: True

Data type name: **Boolean** [1]

Data type description: **The value is True (or could be False)** [1]

YOUR NOTES



Variables & Constants

YOUR NOTES

**Variables & Constants**

Variables and constants are used to store data in a program. Variables can be changed during program execution while constants remain the same.

Declaring Variables and Constants

- Variables are declared using a data type, a name and a value (optional)
- Constants are declared using the 'const' keyword, a name and a value
- In all programming languages, variable names should follow certain rules, such as starting with a letter and not containing spaces
- Examples of data types include integer, float, boolean, and string

Examples in Pseudocode:

Declare a variable called 'score' with a value of 10

```
score ← 10
```

Declare a constant called 'PI' with a value of 3.14

```
const PI ← 3.14
```

Examples in Python:

Declare a variable called 'score' with a value of 10

```
score = 10
```

Declare a constant called 'PI' with a value of 3.14

```
PI = 3.14
```

Examples in Java:

Declare a variable called 'score' with a value of 10

```
int score = 10;
```

Declare a constant called 'PI' with a value of 3.14

```
final double PI = 3.14;
```

Examples in Visual Basic:

Declare a variable called 'score' with a value of 10

```
Dim score As Integer = 10
```

Declare a constant called 'PI' with a value of 3.14

```
Const PI As Double = 3.14
```

**Exam Tip**

- Exam questions will ask you to write pseudocode statements, rather than in a specific language

**Worked Example**

The variables 'name' and 'age' are used to store data in a program:

- name stores the user's name
- age stores the user's age

Write pseudocode statements to declare the variables name and age.

[2]

```
DECLARE name : STRING [1]  
DECLARE age : INTEGER [1]
```

YOUR NOTES



Input & Output

YOUR NOTES



Output

Output refers to the process of displaying or saving the results of a program to the user.

Pseudocode example:

```
OUTPUT "Hello, ", name
```

Python example:

```
print("Hello, ", name)
```

Java example:

```
System.out.println("Hello, " + name);
```

Visual Basic example:

```
Console.WriteLine("Hello, " & name)
```

Input

Input refers to the process of providing data or information to a program.

'User Input' is data or information entered by the user during program execution.

Pseudocode example:

```
OUTPUT "Enter your name"
```

```
INPUT name
```

Python example:

```
name = input("Enter your name: ")
```

Java example:

```
Scanner input = new Scanner(System.in);
```

```
System.out.print("Enter your name: ");
```

```
String name = input.nextLine();
```

Visual Basic example:

```
Console.WriteLine("Enter your name")
```

```
name = Console.ReadLine()
```



Exam Tip

- Remember to prompt the user for input clearly, and format the output to make it readable and understandable.

Sequence

YOUR NOTES



Sequence

What is Sequence?

- Sequence is a concept that involves executing instructions in a particular order
- It is used in programming languages to perform tasks in a step-by-step manner
- Sequence is a fundamental concept in computer science and is used in many programming languages

Pseudocode example:

```
PRINT "Hello, World!"
```

```
x ← 5
```

```
y ← 10
```

```
z ← x + y
```

```
PRINT z
```

Python example:

```
print("Hello, World!")
```

```
x = 5
```

```
y = 10
```

```
z = x + y
```

```
print(z)
```

Java example:

```
System.out.println("Hello, World!");
```

```
int x = 5;
```

```
int y = 10;
```

```
int z = x + y;
```

```
System.out.println(z);
```

Visual Basic example:

```
Console.WriteLine("Hello, World!")
```

```
Dim x As Integer = 5
```

```
Dim y As Integer = 10
```

```
Dim z As Integer = x + y
```

```
Console.WriteLine(z)
```

**Exam Tip**

- Always remember to write your instructions in the order in which you want them to be executed

YOUR NOTES



Selection

YOUR NOTES

**Selection**

What is selection?

Selection is a programming concept that allows you to execute different sets of instructions based on certain conditions. There are two main types of selection statements: IF statements and CASE statements.

If Statements

IF statements allow you to execute a set of instructions if a condition is true. They have the following syntax:

IF condition

THEN

instructions

ENDIF

Pseudocode example:

```
x ← 5
```

```
IF x > 0
```

```
    THEN
```

```
        PRINT "x is positive"
```

```
ENDIF
```

Python example:

```
x = 5
```

```
if x > 0:
```

```
    print("x is positive")
```

Java example:

```
int x = 5;
```

```
if (x > 0) {
```

```
    System.out.println("x is positive");
```

```
}
```

Visual Basic example:

```
Dim x As Integer = 5
```

```
If x > 0 Then
```

```
    Console.WriteLine("x is positive")
```

```
End If
```

IF ELSE Statements

If else statements are used to execute one set of statements if a condition is true and a different set of statements if the condition is false. They have the following syntax:

YOUR NOTES



YOUR NOTES



IF condition

THEN

Instructions

ELSE

Instructions

ENDIF

Pseudocode example:

`x ← 5``IF x > 0``THEN``PRINT "x is positive"``ELSE``PRINT "x is negative"``ENDIF`

Python example:

`x = 5``if x > 0:``print("x is positive")``else:``print("x is negative")`

Java example:

`int x = 5;``if (x > 0) {``System.out.println("x is positive");``} else {``System.out.println("x is negative");``}`

Visual Basic example:

`Dim x As Integer = 5``If x > 0 Then`



```
Console.WriteLine("x is positive")
```

```
Else
```

```
Console.WriteLine("x is negative")
```

```
End If
```

IF ELSE IF Statements

If else if statements are used to test multiple conditions and execute different statements for each condition. They have the following syntax:

```
IF condition
```

```
THEN
```

```
Instructions
```

```
ELSE IF condition
```

```
THEN
```

```
Instructions
```

```
ELSE
```

```
Instructions
```

```
ENDIF
```

Pseudocode example:

```
x ← 5
```

```
IF x > 0
```

```
THEN
```

```
PRINT "x is positive"
```

```
ELSE IF x < 0
```

```
THEN
```

```
PRINT "x is negative"
```

```
ELSE
```

```
PRINT "x is 0"
```

```
ENDIF
```

Python example:

```
x = 5
```

```
if x > 0:
```

```
print("x is positive")
```

```
elif x < 0:
```

```
    print("x is negative")
```

```
else:
```

```
    print("x is 0")
```

Java example:

```
int x = 5;
```

```
if (x > 0) {
```

```
    System.out.println("x is positive");
```

```
} else if (x < 0) {
```

```
    System.out.println("x is negative");
```

```
} else {
```

```
    System.out.println("x is 0");
```

```
}
```

Visual Basic example:

```
Dim x As Integer = 5
```

```
If x > 0 Then
```

```
    Console.WriteLine("x is positive")
```

```
ElseIf x < 0 Then
```

```
    Console.WriteLine("x is negative")
```

```
Else
```

```
    Console.WriteLine("x is 0")
```

```
End If
```

YOUR NOTES





Worked Example

Write an algorithm using pseudocode that:

- Inputs 3 numbers
- Outputs the largest of the three numbers

[3]

INPUT a, b, c

IF a > b AND a > c THEN PRINT a [1]

ELSE IF b > c THEN PRINT b [1]

ELSE PRINT c [1]

YOUR NOTES



Case Statements

CASE statements allow you to execute different sets of instructions based on the value of a variable. They have the following syntax:

CASE OF variable

value1: instructions

value2: instructions

...

OTHERWISE instructions

END CASE

Pseudocode example:

CASE OF number

1: PRINT "Monday"

2: PRINT "Tuesday"

3: PRINT "Wednesday"

4: PRINT "Thursday"

5: PRINT "Friday"

6: PRINT "Saturday"

7: PRINT "Sunday"

OTHERWISE PRINT "Invalid number"

END CASE

Python example:

match (number)

case 1:

print "Monday";

case 2:

print "Tuesday";

case 3:

print "Wednesday";

case 4:

print "Thursday";

YOUR NOTES



YOUR NOTES



```
case 5:
    print "Friday";

case 6:
    print "Saturday";

case 7:
    print "Sunday";

case _:
    print "Invalid number";
```

Java example:

```
switch (number) {

    case 1:
        return "Monday";

    case 2:
        return "Tuesday";

    case 3:
        return "Wednesday";

    case 4:
        return "Thursday";

    case 5:
        return "Friday";

    case 6:
        return "Saturday";

    case 7:
        return "Sunday";

    default:
        return "Invalid number";

}
```

Visual Basic example:

Select Case number

Case 1



Return "Monday"

Case 2

Return "Tuesday"

Case 3

Return "Wednesday"

Case 4

Return "Thursday"

Case 5

Return "Friday"

Case 6

Return "Saturday"

Case 7

Return "Sunday"

Case Else

Return "Invalid number"

End Select



Exam Tip

- Make sure to include all necessary components in the selection statement: the condition, the statement(s) to execute when the condition is true, and any optional statements to execute when the condition is false
- Use proper indentation to make the code easier to read and understand
- Be careful with the syntax of the programming language being used, as it may differ slightly between languages
- Make sure to test the selection statement with various input values to ensure that it works as expected

Iteration

YOUR NOTES



Iteration

What is iteration?

Iteration is the process of repeating a set of instructions until a specific condition is met. It is an important programming concept and is used to automate repetitive tasks.

There are three main types of iteration:

- Count-controlled loops
- Precondition loops
- Postcondition loops

Count-controlled Loops

- A count-controlled loop is used when the number of iterations is known beforehand
- It is also known as a definite loop
- It uses a counter variable that is incremented or decremented after each iteration
- The loop continues until the counter reaches a specific value

Example in Pseudocode:

```
count ← 1
FOR i <= 10
    OUTPUT count
NEXT i
```

Example in Python:

```
for count in range(1, 11):
    print(count)
```

Example in Java:

```
for(int count=1; count<=10; count++){
    System.out.println(count);
}
```

Example in Visual Basic:

```
For count = 1 To 10
    Console.WriteLine(count)
Next count
```

Pre-condition Loops

- A precondition loop is used when the number of iterations is not known beforehand and is dependent on a condition being true
- It is also known as an indefinite loop
- The loop will continue to execute while the condition is true and will stop once the condition becomes false

Example in Pseudocode:

INPUT temperature

WHILE temperature > 37 DO

 OUTPUT "Patient has a fever"

 INPUT temperature

END WHILE

Example in Python:

```
temperature = float(input("Enter temperature: "))
```

```
while temperature > 37:
```

```
    print("Patient has a fever")
```

```
    temperature = float(input("Enter temperature: "))
```

Example in Java:

```
Scanner input = new Scanner(System.in);
```

```
float temperature = input.nextFloat();
```

```
while (temperature > 37) {
```

```
    System.out.println("Patient has a fever");
```

```
    temperature = input.nextFloat();
```

```
}
```

Example in Visual Basic:

```
temperature = InputBox("Enter temperature")
```

```
Do While temperature > 37
```

```
    Console.WriteLine( "Patient has a fever")
```

```
    Console.WriteLine("Enter temperature")
```

```
    temperature=Console.ReadLine()
```

```
Loop
```

YOUR NOTES



Postcondition Loops

- A post-condition loop is used when the loop must execute at least once, even if the condition is false from the start
- The condition is checked at the end of the loop

Example in Pseudocode:

REPEAT

 INPUT guess

UNTIL guess = 42

Example in Python:

Postcondition loops don't exist in Python and would need to be restructured to a precondition loop

Example in Java:

```
Scanner input = new Scanner(System.in);
```

```
int guess = 0;
```

```
do {
```

```
    System.out.print("Enter guess: ");
```

```
    guess = input.nextInt();
```

```
} while (guess != 42);
```

Example in Visual Basic:

Do

```
    Console.WriteLine("Enter guess")
```

```
    guess = Console.ReadLine()
```

Loop Until guess = 42

YOUR NOTES



Totalling & Counting

Totalling

- Totalling involves adding up values, often in a loop
- A total variable can be initialised to 0 and then updated within a loop, such as:

Pseudocode example:

```
total ← 0

for i ← 1 to 10

    input num

    total ← total + num

next i

output total
```

Python example:

```
total = 0

for i in range(1, 11):

    num = int(input("Enter a number: "))

    total += num

print("Total:", total)
```

Java example:

```
int total = 0;

Scanner scanner = new Scanner(System.in);

for (int i = 1; i <= 10; i++) {

    System.out.print("Enter a number: ");

    int num = scanner.nextInt();

    total += num;

}

System.out.println("Total: " + total);
```

Visual Basic example:

```
Dim total As Integer = 0

For i As Integer = 1 To 10

    Console.Write("Enter a number: ")

    Console.ReadLine()
```

YOUR NOTES



```
Dim num As Integer = Integer.Parse(Console.ReadLine())  
  
total += num  
  
Next i  
  
Console.WriteLine("Total: " & total)
```

YOUR NOTES



Counting

- Counting involves keeping track of the number of times a particular event occurs
- A count variable can be initialised to 0 and then updated within a loop, such as:

Pseudocode example:

```
count ← 0

for i ← 1 to 10

    input num

    if num > 5 then

        ount ← count + 1

    end if

next i

output count
```

Python example:

```
count = 0

for i in range(1, 11):

    num = int(input("Enter a number: "))

    if num > 5:

        count += 1

print("Count:", count)
```

Java example:

```
int count = 0;

Scanner scanner = new Scanner(System.in);

for (int i = 1; i <= 10; i++) {

    System.out.print("Enter a number: ");

    int num = scanner.nextInt();

    if (num > 5) {

        count++;

    }

}

System.out.println("Count: " + count);
```

YOUR NOTES



Visual Basic example:

```
Dim count As Integer = 0
```

```
For i As Integer = 1 To 10
```

```
    Console.WriteLine("Enter a number: ")
```

```
    Dim num As Integer = Integer.Parse(Console.ReadLine())
```

```
    If num > 5 Then
```

```
        count += 1
```

```
    End If
```

```
Next i
```

```
Console.WriteLine("Count: " & count)
```



Worked Example

Write an algorithm using pseudocode that:

- Inputs 20 numbers
- Outputs how many of these numbers are greater than 50

[3]

```
FOR x ← 1 TO 20
```

[1]

```
    INPUT Number
```

```
    IF Number > 50 THEN Total ← Total + 1
```

[1]

```
NEXT x
```

```
PRINT total
```

[1]

YOUR NOTES



String Handling

YOUR NOTES

**String Handling**

- Strings are a sequence of characters, such as letters, numbers, and symbols
- They are used to represent text in a computer program
- String handling refers to the various operations that can be performed on strings

Length

- The length of a string is the number of characters it contains
- In Python, the `len()` function is used to find the length of a string
- In Java, the `length()` method is used to find the length of a string
- In Visual Basic, the `Len()` function is used to find the length of a string

Substring

- A substring is a portion of a string
- In Python, the substring can be obtained using the slicing operator `[:]`
- In Java, the `substring()` method is used to obtain a substring
- In Visual Basic, the `Mid()` function is used to obtain a substring

Upper

- The `upper()` method converts all characters of a string to uppercase
- In Python, the `upper()` method is used to convert a string to uppercase
- In Java, the `toUpperCase()` method is used to convert a string to uppercase
- In Visual Basic, the `UCase()` function is used to convert a string to uppercase

Lower

- The `lower()` method converts all characters of a string to lowercase
- In Python, the `lower()` method is used to convert a string to lowercase
- In Java, the `toLowerCase()` method is used to convert a string to lowercase
- In Visual Basic, the `LCase()` function is used to convert a string to lowercase

Pseudocode example:

```
string phrase ← "Save my exams"

length ← LENGTH(phrase)

substring ← SUBSTRING(phrase, 5, 2)

upper ← TO_UPPER(phrase)

lower ← TO_LOWER(phrase)
```

Python example:

```
phrase = "Save my exams"

length = len(phrase)

substring = phrase[4:6]

upper = phrase.upper()
```

```
lower = phrase.lower()
```

Java example:

```
String phrase = "Save my exams";

int length = phrase.length();

String substring = phrase.substring(4, 6);

String upper = phrase.toUpperCase();

String lower = phrase.toLowerCase();
```

Visual basic example:

```
Dim phrase As String = "Save my exams"

Dim length As Integer = phrase.Length

Dim substring As String = phrase.Substring(4, 2)

Dim upper As String = phrase.ToUpper()

Dim lower As String = phrase.ToLower()
```

Position of First Character

- The first character of a string can be at position zero or one, depending on the programming language
- In Python and Java, the first character of a string is at position zero
- In Visual Basic, the first character of a string is at position one

YOUR NOTES





Worked Example

The function `Length(x)` finds the length of a string `x`. The function `substring(x,y,z)` finds a substring of `x` starting at position `y` and `z` characters long. The first character in `x` is in position 1.

Write pseudocode statements to:

- Store the string "Save my exams" in `x`
- Find the length of the string and output it
- Extract the word exams from the string and output it

[6]

`X ← "Save my exams"`

[1] for storing string in `X`.

`OUTPUT Length(X)`

[1] for calling the function `length`. [1] for using the correct parameter `X`.

`Y ← 9`

`Z ← 4`

[1] for using the `substring` function. [1] for correct parameters.

`OUTPUT SubString(X,Y,Z)`

[1] for outputting length and substring return values.



Exam Tip

- Remember that the length of a string is not the same as the index of the last character in the string
- Be careful when counting positions in a string, as the first character can be at position zero or one depending on the programming language

Arithmetic, Logical & Boolean Operators

YOUR NOTES

**Arithmetic Operators**

- Addition (+): Adds two values together
- Subtraction (-): Subtracts one value from another
- Division (/): Divides one value by another
- Multiplication (*): Multiplies two values together
- Exponentiation (^): Raises a number to a power
- Modulo (MOD): Returns the remainder of a division operation
- Integer Division (DIV): Returns the whole number of a division operation

Pseudocode example:

```
a ← 5
b ← 3
c ← a + b
d ← a - b
e ← a * b
f ← a / b
g ← a MOD b
h ← a ^ b
i ← a DIV b
```

Python example:

```
a = 5
b = 3
c = a + b
d = a - b
e = a * b
f = a / b
g = a % b
h = a ** b
i = a // b
```

Java example:

```
int a = 5;
int b = 3;
```

```
int c = a + b;
```

```
int d = a - b;
```

```
int e = a * b;
```

```
double f = (double) a / b;
```

```
int g = a % b;
```

```
double h = Math.pow(a, b);
```

```
int i = a / b;
```

Visual Basic example:

```
Dim a As Integer = 5
```

```
Dim b As Integer = 3
```

```
Dim c As Integer = a + b
```

```
Dim d As Integer = a - b
```

```
Dim e As Integer = a * b
```

```
Dim f As Double = a / b
```

```
Dim g As Integer = a Mod b
```

```
Dim h As Double = Math.Pow(a, b)
```

```
Dim i As Integer = a \ b
```

YOUR NOTES



Logical Operators

- Equal to (=): Returns true if two values are equal
- Less than (<): Returns true if the first value is less than the second value
- Less than or equal to (<=): Returns true if the first value is less than or equal to the second value
- Greater than (>): Returns true if the first value is greater than the second value
- Greater than or equal to (>=): Returns true if the first value is greater than or equal to the second value
- Not equal to (<>): Returns true if two values are not equal

Pseudocode example:

a ← 5

b ← 3

c ← (a = b)

d ← (a < b)

e ← (a <= b)

f ← (a > b)

g ← (a >= b)

h ← (a <> b)

Python example:

a = 5

b = 3

c = (a == b)

d = (a < b)

e = (a <= b)

f = (a > b)

g = (a >= b)

h = (a != b)

Java example:

int a = 5;

int b = 3;

boolean c = (a == b);

boolean d = (a < b);

boolean e = (a <= b);

YOUR NOTES



```
boolean f = (a > b);
```

```
boolean g = (a >= b);
```

```
boolean h = (a != b);
```

Visual Basic example:

```
Dim a As Integer = 5
```

```
Dim b As Integer = 3
```

```
Dim c As Boolean = (a = b)
```

```
Dim d As Boolean = (a < b)
```

```
Dim e As Boolean = (a <= b)
```

```
Dim f As Boolean = (a > b)
```

```
Dim g As Boolean = (a >= b)
```

```
Dim h As Boolean = (a <> b)
```

YOUR NOTES



Boolean Operators

Boolean operators are logical operators that can be used to compare two or more values and return a Boolean value (True or False) based on the comparison. There are 3 you need to know:

- AND: Returns True if both conditions are True
- OR: Returns True if one or both conditions are True
- NOT: Returns the opposite of the condition (True if False, False if True)

Boolean Operators in Pseudocode:

```
IF (condition1 AND condition2) THEN
```

```
// code to execute if both conditions are True
```

```
END IF
```

```
IF (condition1 OR condition2) THEN
```

```
// code to execute if one or both conditions are True
```

```
END IF
```

```
IF NOT(condition) THEN
```

```
// code to execute if the condition is False
```

```
END IF
```

Boolean Operators in Python:

```
if condition1 and condition2:
```

```
# code to execute if both conditions are True
```

```
if condition1 or condition2:
```

```
# code to execute if one or both conditions are True
```

```
if not condition:
```

```
# code to execute if the condition is False
```

Boolean Operators in Java:

```
if (condition1 && condition2) {
```

```
// code to execute if both conditions are True
```

```
}
```

```
if (condition1 || condition2) {
```

```
// code to execute if one or both conditions are True
```

```
}
```

```
if (!condition) {
```

YOUR NOTES



```
// code to execute if the condition is False
```

```
}
```

Boolean Operators in Visual Basic:

```
If condition1 And condition2 Then
```

```
' code to execute if both conditions are True
```

```
End If
```

```
If condition1 Or condition2 Then
```

```
' code to execute if one or both conditions are True
```

```
End If
```

```
If Not condition Then
```

```
' code to execute if the condition is False
```

```
End If
```



Exam Tip

- Boolean operators are often used in conditional statements such as if, while, and for loops
- Boolean operators can be combined with comparison operators such as == (equal to), != (not equal to), < (less than), > (greater than), <= (less than or equal to), and >= (greater than or equal to)
- Be careful when using the NOT operator, as it can sometimes lead to unexpected results. It is always a good idea to test your code with different inputs to ensure that it works as expected

YOUR NOTES



Nested Statements

Nested Statements

- Nested statements involve including one statement within another statement. This can be done with both selection (if/else) and iteration (for/while) statements
- In programming languages like Python, Java, and Visual Basic, nested statements are often indicated by indenting the inner statement(s) relative to the outer statement(s)

**Exam Tip**

- You will not be required to write more than three levels of nested statements

YOUR NOTES



Nested Selection

- Nested selection is an if statement inside an if statement
- If the first if statement is true, it will run the if statement which is nested inside
- Otherwise, it will skip to the "else if" or "else" which is part of that if statement

Pseudocode example:

```
if a > b then
    if b > c then
        output "a is the largest"
    else
        output "c is the largest"
else
    if a > c then
        output "b is the largest"
    else
        output "c is the largest"
```

Python example:

```
if a > b:
    if b > c:
        print("a is the largest")
    else:
        print("c is the largest")
else:
    if a > c:
        print("b is the largest")
    else:
        print("c is the largest")
```

Java example:

```
if (a > b) {
    if (b > c) {
        System.out.println("a is the largest");
    } else {
```

YOUR NOTES



```
        System.out.println("c is the largest");
    }
} else {
    if (a > c) {
        System.out.println("b is the largest");
    } else {
        System.out.println("c is the largest");
    }
}
```

Visual Basic example:

```
If a > b Then
    If b > c Then
        Console.WriteLine("a is the largest")
    Else
        Console.WriteLine("c is the largest")
    End If
Else
    If a > c Then
        Console.WriteLine("b is the largest")
    Else
        Console.WriteLine("c is the largest")
    End If
End If
```



Exam Tip

- Indentation is key so make sure it's clear in your answer which if statement line is part of which. It's more clear when you use end if in the appropriate places, too.

YOUR NOTES



Nested Iteration

Nested iteration refers to a loop inside another loop.

Pseudocode example:

```
FOR i ← 1 TO 10  
    FOR j ← 1 TO 5  
        OUTPUT "i = ", i, " j = ", j  
    END FOR  
END FOR
```

Python example:

```
for i in range(1, 11):  
    for j in range(1, 6):  
        print("i = ", i, " j = ", j)
```

Java example:

```
for (int i = 1; i <= 10; i++) {  
    for (int j = 1; j <= 5; j++) {  
        System.out.println("i = " + i + " j = " + j);  
    }  
}
```

Visual Basic example:

```
For i As Integer = 1 To 10  
    For j As Integer = 1 To 5  
        Console.WriteLine("i = " & i & " j = " & j)  
    Next j  
Next i
```



Exam Tip

- Nested iteration is useful when we need to perform a task for a specific range of values
- It is important to keep track of the number of nested statements
- It is important to keep nested statements organized and clear. Use consistent indentation and avoid going too many levels deep, as this can make the code difficult to read and understand

YOUR NOTES



Procedures & Functions

YOUR NOTES



Subroutines

Procedures and functions are types of subroutines. They are a sequence of instructions that perform a specific task or set of tasks

- Subroutines are often used to simplify a program by breaking it into smaller, more manageable parts. They can be used to:
 - Avoid duplicating code and can be reused throughout a program,
 - Improve the readability and maintainability of code,
 - Perform calculations, to retrieve data, or to make decisions based on input.
- Parameters are values that are passed into a subroutine. They can:
 - Be used to customise the behaviour of a subroutine,
 - Be of different types,
 - Have default values, which are used if no value is passed in,
 - Be passed by value or by reference.
 - When a parameter is passed by value, a copy of the parameter is made and used within the subroutine
 - When a parameter is passed by reference, the original parameter is used within the subroutine, and any changes made to the parameter will be reflected outside of the subroutine
- Subroutines can have multiple parameters

What's the difference between a procedure and function?

Functions return a value whereas a procedure does not.

Procedures

- Procedures are defined using the PROCEDURE keyword in pseudocode, def keyword in Python, and Sub keyword in Visual Basic and Java
- Procedures can be called from other parts of the program using their name

Pseudocode example:

```
PROCEDURE calculate_area(length: INTEGER, width: INTEGER)
```

```
    area ← length * width
```

```
    OUTPUT "The area is " + area
```

```
END PROCEDURE
```

To call the procedure you would use the following pseudocode:

```
calculate_area(5,3)
```

Python example:

```
def calculate_area(length, width):
```

```
    area = length * width
```

```
    print("The area is ", area)
```

```
calculate_area(5,3)
```

Java example:

```
public static void calculateArea(int length, int width) {
```

```
    int area = length * width;
```

```
    System.out.println("The area is " + area);
```

```
}
```

```
calculateArea(5, 3);
```

Visual Basic example:

```
Sub CalculateArea(length As Integer, width As Integer)
```

```
    Dim area As Integer = length * width
```

```
    Console.WriteLine("The area is " & area)
```

```
End Sub
```

```
CalculateArea(5, 3)
```

YOUR NOTES



Functions

- Functions are defined using the FUNCTION keyword in pseudocode, def keyword in Python, and Function keyword in Visual Basic and Java
- Functions return a value using the RETURN keyword in pseudocode, return statement in Python, and function name in Visual Basic and Java
- Functions can be called from other parts of the program using their name, and their return value can be stored in a variable

Pseudocode example:

```
PROCEDURE calculate_area(length: INTEGER, width: INTEGER)
```

```
    area ← length * width
```

```
    RETURN area
```

```
END PROCEDURE
```

To output the value returned from the function you would use the following pseudocode:

```
OUTPUT(calculate_area(5,3))
```

Python example:

```
def calculate_area(length, width):
```

```
    area = length * width
```

```
    return area
```

```
print(calculate_area(5,3))
```

Java example:

```
public static void calculateArea(int length, int width) {
```

```
    int area = length * width;
```

```
    return area;
```

```
}
```

```
System.out.println(calculateArea(5, 3));
```

Visual Basic example:

```
CalculateArea(5, 3)
```

```
Sub CalculateArea(length As Integer, width As Integer)
```

```
    Dim area As Integer = length * width
```

```
    Return area
```

```
End Sub
```

YOUR NOTES



Local & Global Variables

YOUR NOTES

**Local & Global Variables**

- A local variable is a variable that is declared inside a subroutine and can only be accessed from within that subroutine
- A global variable is a variable that is declared outside of a subroutine and can be accessed from anywhere in the program
- Global variables can be used to pass data between different subroutine
- Overuse of global variables can make a program difficult to understand and debug and is therefore discouraged as it's considered bad practice

Pseudocode example:`global total_score``FUNCTION calculate_average(num1: INTEGER, num2: INTEGER)``average ← (num1 + num2) / 2``RETURN average``END FUNCTION``total_score ← 0``PROCEDURE add_score(score: INTEGER)``total_score ← total_score + score``END PROCEDURE`**Python example:**`def calculate_average(num1, num2):``average = (num1 + num2) / 2``return average``total_score = 0``def add_score(score):``global total_score``total_score = total_score + score`**Java example:**`static int totalScore = 0;``public static void main(String args[]) {``double average = calculateAverage(5, 7);``System.out.println("Average: " + average);`

```
addScore(10);

System.out.println("Total score: " + totalScore);

}

public static double calculateAverage(int num1, int num2) {

    double average = (num1 + num2) / 2.0;

    return average;

}

public static void addScore(int score) {

    totalScore = totalScore + score;

}
```

Visual Basic example:

```
Dim totalScore As Integer = 0

Dim average As Double = CalculateAverage(5, 7)

Console.WriteLine("Average: " & average)

AddScore(10)

Console.WriteLine("Total score: " & totalScore)

Function CalculateAverage(num1 As Integer, num2 As Integer) As Double

    Dim average As Double = (num1 + num2) / 2.0

    Return average

End Function

Sub AddScore(score As Integer)

    totalScore = totalScore + score

End Sub
```

YOUR NOTES



Library Routines

YOUR NOTES



Library Routines

Library routines are pre-written code that can be used in programs to perform specific tasks.

- Using library routines saves time by not having to write code from scratch
- Library routines are also tested and proven to work, so errors are less likely
- Some commonly used library routines include:
 - Input/output routines
 - Maths routines
- Libraries can be included in programs by importing them
- The specific syntax for importing a library may vary depending on the programming language being used
- It is important to check the documentation for a library to understand its usage and available functions

Maths routines

- MOD: a function that returns the remainder when one number is divided by another number
 - Pseudocode example: `x MOD y`
 - Python example: `x % y`
 - Java example: `x % y`
 - Visual Basic example: `x Mod y`
- DIV: a function that returns the quotient when one number is divided by another number
 - Pseudocode example: `x DIV y`
 - Python example: `x // y`
 - Java example: `x / y`
 - Visual Basic example: `x \ y`
- ROUND: a function that rounds a number to a specified number of decimal places
 - Pseudocode example: `ROUND(x, n)`
 - Python example: `round(x, n)`
 - Java example: `Math.round(x * 10^n) / x;`
 - Visual Basic example: `Math.Round(x, n)`
- RANDOM: a function that generates a random number between x and n
 - Pseudocode example: `RANDOM(x,n)`
 - Python example: `random.randint(x,n)`
 - Java example: `rand.nextInt(x)`
 - Visual Basic example: `rand.next(x,n)`



Exam Tip

- Remember to import or include the appropriate library before using the routines in your code

Maintaining Programs

YOUR NOTES

**Maintaining Programs****Why is it important to create a maintainable program?**

- Improve program quality:
 - A maintainable program is easier to understand and modify, which leads to fewer bugs and better program quality
- Reduce development time and costs:
 - A maintainable program requires less time and effort to modify, which reduces development time and costs
- Enables collaboration:
 - A maintainable program makes it easier for multiple developers to work together on the same project, as it's easier to understand and modify
- Increase program lifespan:
 - A maintainable program is more likely to be updated and maintained over time, which increases its lifespan and usefulness
- Adapt to changing requirements:
 - A maintainable program is easier to modify to adapt to changing requirements or new features

How do you create a well maintained program?

- Use meaningful identifiers:
 - Identifiers are names given to variables, constants, arrays, procedures and functions
 - Use descriptive and meaningful identifiers to make your code easier to understand and maintain
 - Avoid using single letters or abbreviations that may not be clear to others
- Use the commenting feature provided by the programming language:
 - Comments are used to add descriptions to the code that help readers understand what the code is doing
 - Use comments to explain the purpose of variables, constants, procedures, functions and any other parts of the code that may not be immediately clear
 - Use comments to document any assumptions or limitations of the code
- Use procedures and functions:
 - Procedures and functions are reusable blocks of code that perform specific tasks
 - Using procedures and functions allows you to modularise your code and make it easier to understand, debug and maintain
 - Procedures and functions should have descriptive names that clearly indicate what they do
- Relevant and appropriate commenting of syntax:
 - Commenting of syntax is used to explain the purpose of individual lines or blocks of code within the program
 - Use commenting of syntax to describe complex algorithms, or to provide additional information on the purpose or behaviour of code